

Предиктивное обслуживание оборудования (Predictive Maintenance)

1. Введение

Бизнес-контекст: В условиях современного промышленного производства незапланированные простои оборудования приводят к колоссальным финансовым убыткам. Традиционные подходы к техническому обслуживанию (реактивный — ремонт после поломки, и регламентный — плановый осмотр по расписанию) либо допускают внезапные аварии, либо приводят к преждевременной замене исправных узлов.

Актуальность: Внедрение системы предиктивного технического обслуживания (Predictive Maintenance) на базе машинного обучения позволяет осуществлять непрерывный мониторинг состояния оборудования на основе показаний телеметрических датчиков и прогнозировать вероятность сбоя до его фактического наступления.

Цель проекта: Построить систему, которая предсказывает вероятность выхода оборудования из строя в ближайшие 7 дней на основе данных с датчиков.

Суть задачи заключается в переходе от реактивного ремонта (устранение аварии после остановки) и жесткого планового ТО к обслуживанию по фактическому состоянию оборудования. На основе потока данных с телеметрических датчиков система прогнозирует вероятность выхода оборудования из строя в ближайшие 7 дней.

Основной экономический эффект достигается за счет:

- Минимизации незапланированных простоев (Unplanned Downtime) производственной линии.
- Предотвращения дорогостоящего капитального ремонта путем раннего выявления износа или перегрева.
- Оптимизации работы сервисных бригад и складских запасов комплектующих.

2. Сбор и анализ требований

Бизнес-метрики:

- **Сокращение времени аварийного простоя (Downtime Reduction):** Уменьшение часов незапланированной остановки оборудования на 20–30%.
- **Экономия на ТОиР (Maintenance Cost Savings):** Снижение совокупных расходов на ремонт и запчасти на 15–20%.
- **ROI системы:** Отношение предотвращенного финансового ущерба от аварий к стоимости разработки и содержания ML-инфраструктуры.

ML-метрики:

- Recall не менее 0.85 (критично не пропустить поломку)

- False Positive Rate < 0.15 (минимизация ложных тревог)
- Модель должна работать с данными в режиме реального времени
- Время инференса: < 50ms

Функциональные и нефункциональные требования

Категория	Требование	Критерий приемки
Функциональные	Потоковый сбор телеметрии	Непрерывная приемка показателей: температура воздуха, температура процесса, скорость вращения (rpm), крутящий момент (Torque), износ инструмента (Tool wear)
Функциональные	Расчет признаков (Feature Store)	Вычисление статистических агрегатов (скользящие средние, стандартные отклонения, разности температур) в реальном времени
Функциональные	Скоринг и выдача прогноза	Оценка вероятности поломки на ближайшие 7 дней для каждой единицы оборудования
Функциональные	Алертинг и интеграция с ERP/CMMS	Автоматическое формирование заявки на обслуживание в случае превышения порога риска
Нефункциональные	Время инференса (Latency)	<50 ms на запрос при перцентиле p99
Нефункциональные	Режим работы (Processing)	Real-time / Near Real-Time (задержка потоковой обработки <1 с).
Нефункциональные	Доступность (Availability)	99.9% uptime (допустимый простой менее 8.7 часов в год)
Нефункциональные	Пропускная способность	Не менее 10000 RPS / EPS (events per second) в пиковой нагрузке
Нефункциональные	Масштабируемость	Горизонтальное масштабирование компонентов

3. Архитектура системы

Сформируем архитектурную схему, описывающую полный жизненный цикл данных (от датчиков до формирования заявок на ремонт и мониторинга качества модели). Она разделена на два основных контура: онлайн (Real-time) для мгновенного предсказания и офлайн (Обучение) для хранения истории и тренировки алгоритмов. Схема представлена на Рисунке 1

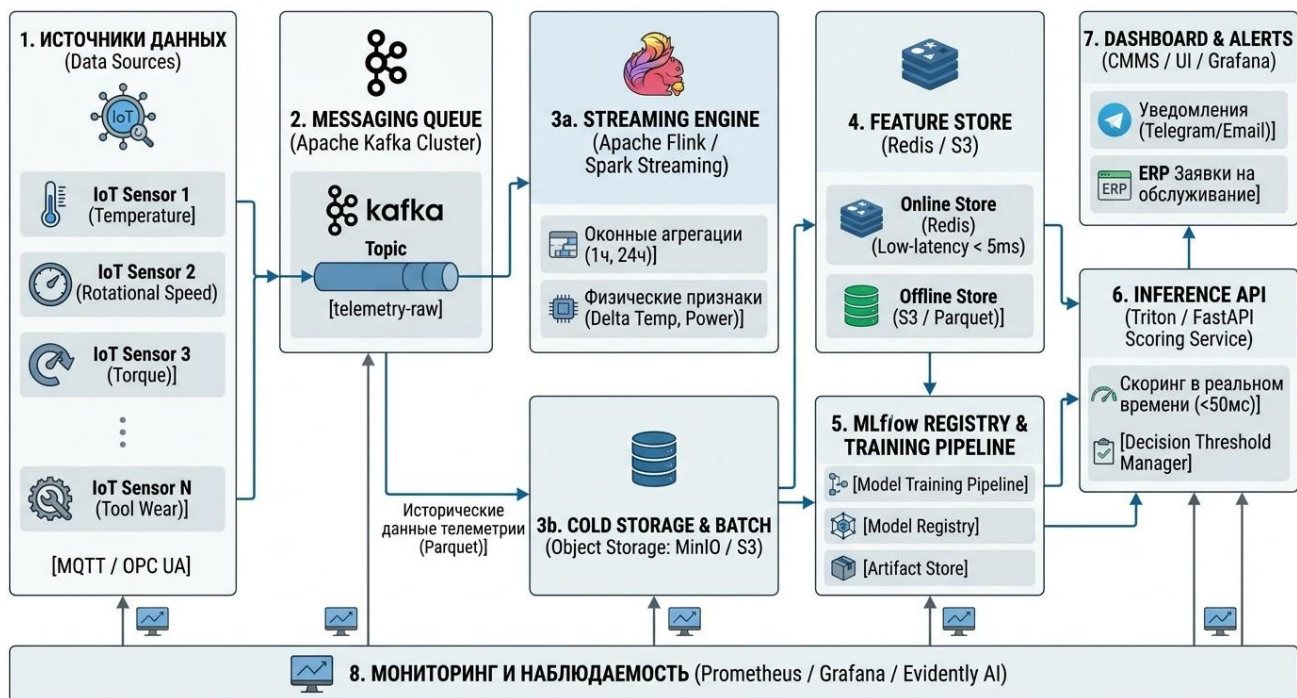


Рисунок 1: Архитектурная схема системы

Данная схема включает следующие компоненты:

1. Источники данных (Data Sources)

- **Что это:** Физические IoT-датчики, установленные непосредственно на производственном оборудовании (станках, турбинах, двигателях).
- **Для чего нужно:** Непрерывно измеряют базовые параметры системы: температуру воздуха и процесса, скорость вращения, крутящий момент и физический износ инструмента.
- **Место в процессе:** Это точка зарождения данных. С помощью промышленных протоколов (MQTT или OPC UA) сырая телеметрия отправляется в облако или на локальный сервер.

2. Брокер сообщений (Messaging Queue)

- **Что это:** Высокопроизводительная шина данных, чаще всего кластер Apache Kafka.
- **Для чего нужно:** Выступает в роли надежного буфера-амортизатора. Если с 10 000 датчиков одновременно придет сигнал, система не упадет. Kafka выстраивает все сырые данные в единую очередь (топик telemetry-raw).
- **Место в процессе:** Принимает данные от датчиков и раздает их (дублирует) сразу двум независимым потребителям: быстрому потоковому процессору (3a) и долговременному хранилищу (3b).

3a. Поточковая обработка (Streaming Engine)

- **Что это:** Движки вычислений в реальном времени (Apache Flink или Spark Streaming).
- **Для чего нужно:** Сырые цифры с датчиков малоприспособны для ML. Этот компонент на лету вычисляет новые признаки (Feature Engineering): например, разницу температур (Delta Temp),

мощность ($\text{Power} = \text{крутящий момент} \times \text{обороты}$), а также оконные агрегации (средняя температура за последний 1 час или 24 часа).

- **Место в процессе:** Забирает сырой поток из Kafka, обогащает его математикой и мгновенно складывает готовые к скорингу признаки в быстрый Feature Store (4).

3b. Холодное хранилище (Cold Storage & Batch)

- **Что это:** Объектное хранилище (S3 AWS, MinIO, Hadoop HDFS), где данные лежат в сжатом столбцовом формате Parquet.
- **Для чего нужно:** Дешевое и надежное хранение петабайтов исторической телеметрии за месяцы и годы работы цеха.
- **Место в процессе:** Данные отсюда не используются для мгновенных предсказаний. Они нужны исключительно Data Scientist'ам для проведения исследований и регулярного переобучения ML-моделей (5).

4. Хранилище признаков (Feature Store)

- **Что это:** Специализированная база данных, разделенная на две части.
- **Online Store (на базе Redis):** Держит в оперативной памяти только самое последнее, актуальное состояние каждого станка. Гарантирует отдачу данных за миллисекунды ($< 5\text{ms}$).
- **Offline Store (на базе S3):** Хранит слепки всех признаков за прошлые периоды для корректного обучения.
- **Для чего нужно:** Гарантирует, что модель при обучении (офлайн) и при работе (онлайн) использует одни и те же математические формулы и данные, исключая расхождения в логике.
- **Место в процессе:** Мост между инженерией данных и машинным обучением.

5. MLflow Registry & Training Pipeline

- **Что это:** Инфраструктура для тренировки и версионирования моделей машинного обучения.
- **Для чего нужно:** Регулярно (например, раз в неделю) запускает пайплайн обучения на новых исторических данных. MLflow выступает в роли "Git для нейросетей" — записывает метрики экспериментов, сохраняет артефакты (веса модели) и помечает самую успешную модель тегом "Production".
- **Место в процессе:** Берет массивы данных из (3b) и (4), тренирует модель и передает готовый файл весов в сервис инференса (6).

6. Inference API (Сервис скоринга)

- **Что это:** Микросервис (Triton Inference Server или самописный на FastAPI), обрабатывающий ML-модель в REST/gRPC API.
- **Для чего нужно:** Это "мозг" системы в реальном времени. Он получает команду предсказать статус конкретного станка, мгновенно запрашивает его текущие физические параметры из Redis (4), прогоняет их через модель и выдает вероятность поломки в течение 7 дней. Встроенный Decision Threshold Manager проверяет, превышает ли вероятность заданный порог отсечения (например, 0.02 для обеспечения требуемого Recall ≥ 0.85). Время отклика строго ограничено ($< 50\text{мс}$).
- **Место в процессе:** Ядро логики, связывающее текущие данные станков с бизнес-уведомлениями (7).

7. Dashboard & Alerts (Бизнес-интерфейсы)

- **Что это:** Пользовательские интерфейсы и интеграционные шлюзы.
- **Для чего нужно:** Превращает сухие вероятности от ML-модели в полезные действия для людей. Если сервис скоринга бьет тревогу, этот блок автоматически создает наряд-запуск на ремонт в корпоративной ERP-системе (например, SAP или 1C) или присылает срочное уведомление главному инженеру в Telegram/Email.
- **Место в процессе:** Финальный этап конвейера, где техническое предсказание приносит реальную бизнес-ценность (предотвращение аварии).

8. Мониторинг и наблюдаемость (Monitoring)

- **Что это:** Инфраструктурные (Prometheus, Grafana) и специализированные ML-инструменты (Evidently AI).
- **Для чего нужно:** Охватывает все слои архитектуры. Следит, чтобы:
 - Серверы не перегружались по памяти/CPU.
 - Очередь сообщений не отставала.
 - Скорость предсказаний (Latency) не превышала 50мс.
- Главное для ML: Следит за деградацией модели (Data Drift). Если физика процесса изменилась (например, поставили новые резцы, и их износ идет иначе), распределение данных меняется. Мониторинг подаст сигнал, что модель ослепла и нужно срочно запускать переобучение в MLflow (5).

4. Анализ данных

4.1. Описание датасета

В исследовании использовался набор данных AI4I 2020 Predictive Maintenance Dataset, содержащий 10,000 записей телеметрии промышленного оборудования. Датасет включает физические параметры процесса: температуру воздуха и процесса (K), скорость вращения (RPM), крутящий момент (Nm) и износ инструмента (мин).

4.2. Анализ распределений

Проверим, как распределены физические показатели: температура, скорость вращения и крутящий момент

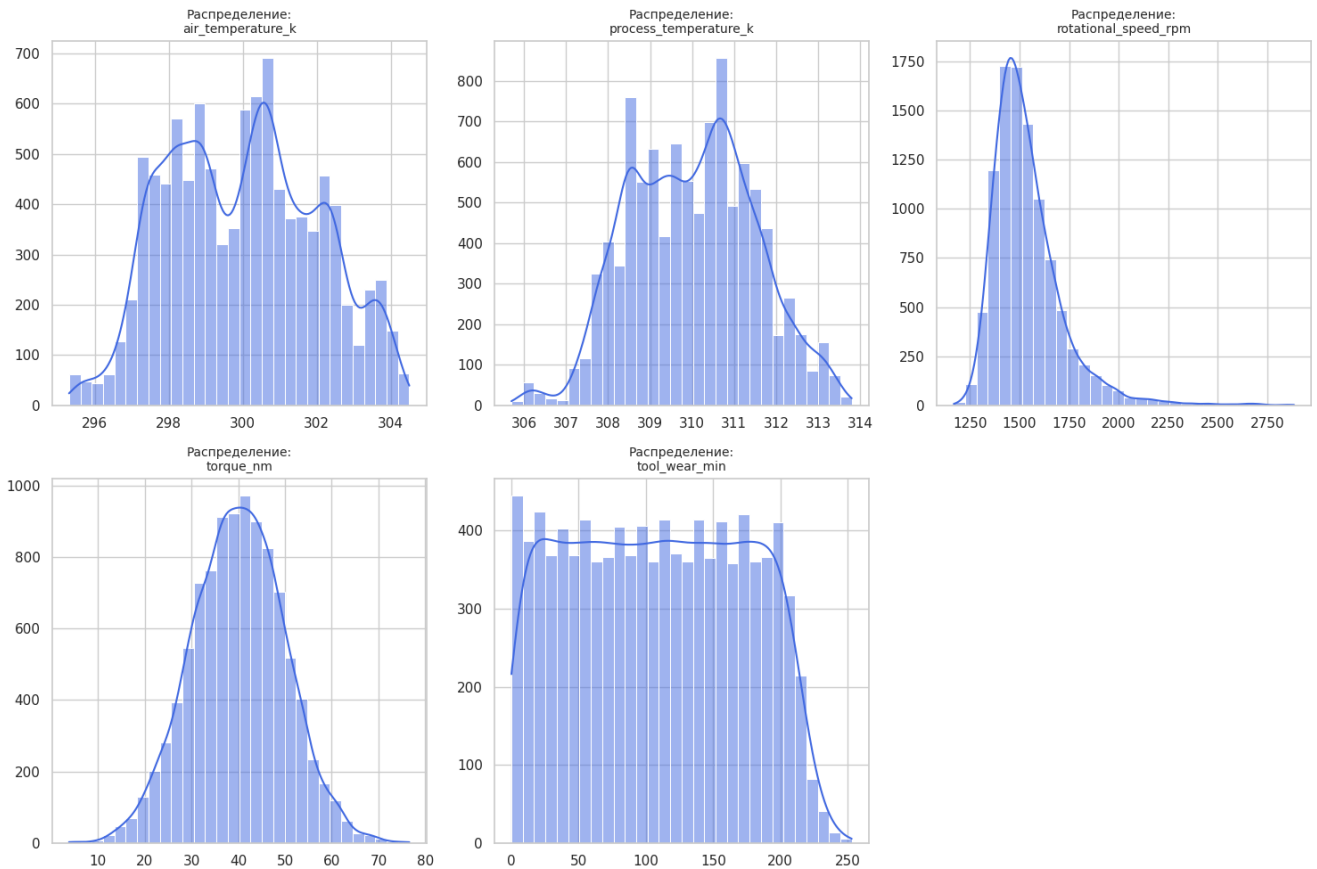


Рисунок 2: Гистограммы распределений показателей

- **Температурные показатели:**

- **air_temperature_k** (Температура воздуха): Распределение имеет сложную мультимодальную форму (наблюдаются явные пики около 298, 300.5 и 302 Кельвинов). Это свидетельствует о том, что внутри данных могут быть скрытые подгруппы — например, замеры проводились в разные смены, в разные сезоны или в цехах с разными условиями.
- **process_temperature_k** (Температура процесса): График также мультимодальный и визуально сильно повторяет форму температуры воздуха, но в более высоком числовом диапазоне (от 307 до 312 Кельвинов). Скорее всего, между этими двумя показателями существует сильная прямая корреляция.

- **Механические характеристики:**

- **rotational_speed_rpm** (Скорость вращения): Здесь наблюдается ярко выраженная правосторонняя асимметрия (скошенность вправо). Основной рабочий режим оборудования составляет примерно 1400–1600 оборотов в минуту, однако присутствует длинный «хвост» из относительно редких, высоких значений (вплоть до 2800 об/мин).
- **torque_nm** (Крутящий момент): Это отличный пример классического нормального (гауссовского) распределения в форме симметричного колокола.

Среднее значение и медиана находятся на отметке около 40 Нм, а основная масса наблюдений сосредоточена в пределах от 20 до 60 Нм.

- **Износ инструмента:** `tool_wear_min` (Время износа инструмента): График демонстрирует практически равномерное распределение. Частота встречаемости любых значений в диапазоне от 0 до 200–220 минут почти одинакова. Это типичная картина для процессов технического обслуживания: инструмент работает с нуля, его наработка равномерно фиксируется, а по достижении определенного лимита (около 230 минут) деталь меняют, и таймер обнуляется.

4.3. Анализ целевой переменной

Выявили проблему существенного дисбаланса классов: всего 3.39% объектов принадлежат к классу поломки (339 случаев из 10 000). Для решения этой проблемы при обучении применялась взвешенная функция потерь (`class_weight='balanced'`) и оптимизация порога вероятности.

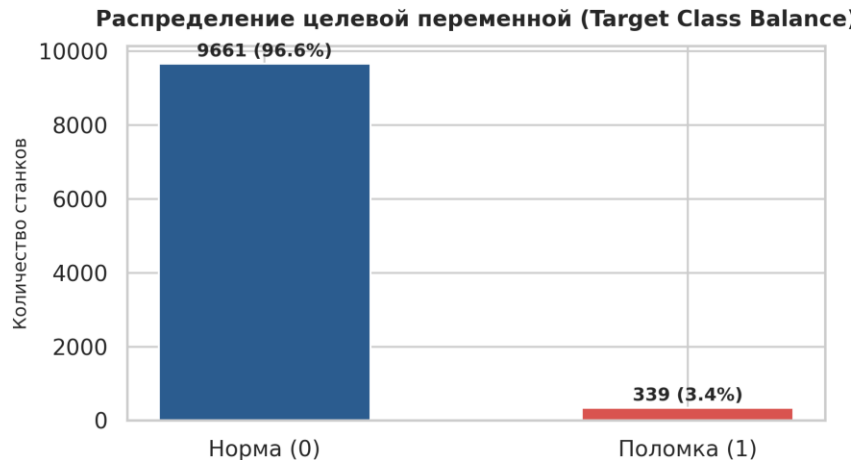


Рисунок 3: Дисбаланс классов в обучающей выборке

4.4. Анализ выбросов

Boxplot отлично показывает аномальные значения, которые сильно выбиваются из общей массы.

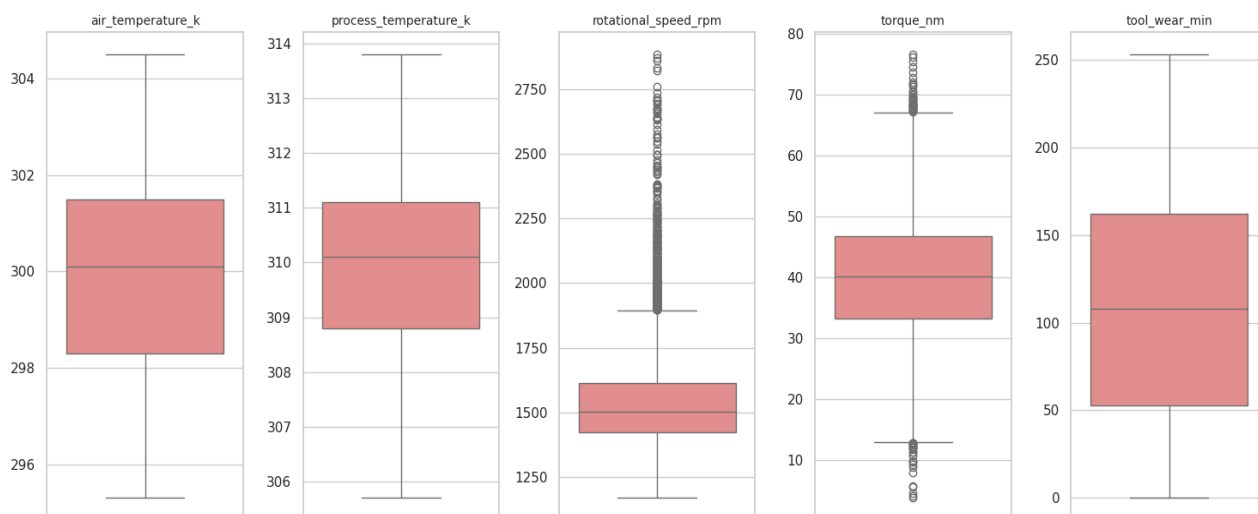


Рисунок 4: Диаграмма Вохplot выбросов показателей

На графике четко видны статистические выбросы (точки выше "усов") для признака Rotational speed [rpm]. Есть единичные аномальные выбросы свыше 2500 rpm. Удалять их нельзя, так как резкое повышение оборотов, вероятно, напрямую связано с перегревом или предвещает поломку оборудования. Эти выбросы мы ограничиваем статистическими границами.

4.5. Анализ корреляций

Данный анализ покажет, какие датчики зависят друг от друга.

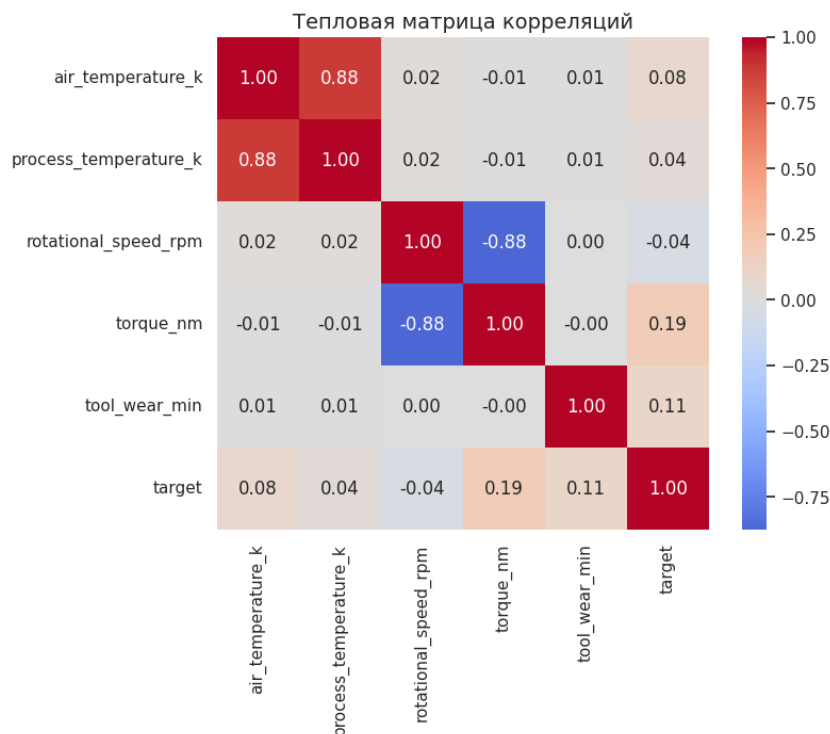


Рисунок 5: Тепловая матрица корреляций признаков

Air_temperature_k и process_temperature_k имеют сильнейшую связь. rotational_speed_rpm и torque_nm имеют сильную обратную связь

4.6. Удаление неинформативных признаков

Колонки UDI (уникальный идентификатор строки) и Product ID (серийный номер) не несут математического смысла для прогнозирования сбоев (они просто нумеруют детали). Оставлять их опасно — модель может просто "запомнить" ID сломавшихся деталей вместо поиска закономерностей (переобучение).

5. Методология

5.1 Генерация признаков

Основываясь на физике процесса (и выводах из матрицы корреляций), мы создадим два новых признака, которые помогут модели:

- Разность температур (Temp_diff): Разница между температурой процесса и воздуха. Она покажет, насколько оборудование нагревает окружающую среду сверх нормы.
 $\text{Temp_Diff} = \text{Process_Temperature} - \text{Air_Temperature}$
- Мощность (Power): Физическая формула мощности: Скорость * Момент. Учитывая их обратную корреляцию, их произведение даст стабильный показатель нагрузки на ось.
 $\text{Power} = \text{Rotational_Speed} * \text{Torque}$

И добавляем бинарный флаг Is_high_wear - критического износа инструмента (более 200 минут)

Эти признаки увеличили разделяющую способность линейных и древесных моделей.

Схема валидации: Данные разбивались в соотношении 80/20 с использованием стратификации (StratifiedKFold, k=5) для сохранения доли редкого целевого класса во всех фолдах.

- **Temp_diff:** Распределение стало бимодальным (два явных пика на 9.5K и 11K). Это очень сильный признак: возможно, одна из "вершин" характеризует пред-аварийный режим работы (сильный нагрев).
- **Power:** Мощность распределена абсолютно нормально со средним значением около 60 000 единиц. Любые сильные отклонения мощности от этого купола вправо или влево будут четким сигналом для алгоритма о механической перегрузке или сбое питания.
- **Итог очистки:** Данные полностью готовы для следующего этапа (разбиения на train/test и обучения Baseline-модели)

5.2. Baseline модель (Базовое решение)

Baseline — это простая, быстро обучаемая модель, которая служит точкой отсчета. Если сложная нейросеть работает хуже базлайна, значит, мы делаем что-то не так. Для задачи бинарной классификации (поломка: да/нет) лучше всего подходят Логистическая регрессия (Logistic Regression)

Особенность: Наша выборка сильно несбалансирована (поломок всего \$3.4). Если обучить модель "в лоб", она будет предсказывать нули (отсутствие поломок) и получит точность \$96.6,

но пропустит все аварии. Поэтому мы обязательно используем параметр `class_weight='balanced'`, который "штрафует" модель за пропуск редкого класса.

Логистическая регрессия находит 88.2% поломок ($\text{Recall} = 0.882$), что удовлетворяет бизнесу. Однако она дает огромное количество ложных тревог: $\text{FPR} = 0.142$ (почти 15% нормальных станков отправляются на ремонт) и очень низкий $\text{Precision} = 0.17$ (из всех вызовов мастера только 17% реально связаны с поломкой).

5.3. Гипотезы для проверки

- **Гипотеза 1 (Feature Engineering & Physical Domain Rules):**
 - *Суть:* Добавление предметных физических признаков — разности температур и механической мощности.
 - *Проверка:* Оценка качества модели на сырых признаках в сравнении с расширенным набором фичей.
- **Гипотеза 2 (Выбор алгоритма и оптимизация инференса):**
 - *Суть:* Использование градиентного бустинга (LightGBM / CatBoost), обеспечит лучший выбор по соотношению качества и скорости работы, обогнав по качеству baseline, а по скорости — глубокие нейросети (LSTM).
 - *Проверка:* Бенчмаркинг скорости инференса при помощи нагрузочного тестирования (1000 RPS) и сравнение F2-Score на кросс-валидации.
- **Гипотеза 3 (Разделение по типам сбоев):**
 - *Суть:* Обучение отдельных бинарных классификаторов под специфические типы сбоев, что даст более высокий Recall по редким типам сбоев, чем единый бинарный классификатор

5.3. Эксперименты с алгоритмами (Сложные модели)

Проведем обучение следующих алгоритмов:

- Random Forest (Случайный лес): Сотни деревьев решений голосуют за результат.
- Gradient Boosting (Бустинг): Деревья строятся по очереди, каждое следующее исправляет ошибки предыдущего.
- SVM (Метод опорных векторов): Ищет сложную математическую границу между классами.
- Neural Network (Нейросеть): Имитирует работу мозга (многослойный перцептрон).

И применим Кросс-валидацию — оценку модели на разных кусочках данных для надежности. И для Бустинга проведем гиперпараметрическую оптимизацию с помощью случайного поиска лучших настроек (RandomizedSearchCV)

6. Результаты и оценка моделей

6.1. Сравнение алгоритмов: Было проведено сравнение четырех алгоритмов машинного обучения. Данные сведены в таблицу:

=== Сравнение моделей на Тестовой выборке ===

	ROC-AUC	Recall (Находит аварии)	Precision (Точность)	F1-Score
Модель				
Baseline (LogReg)	0.882201	0.897059	0.141860	0.244980
Random Forest	0.941173	0.485294	0.750000	0.589286
Tuned Gradient Boosting	0.978634	0.985294	0.272358	0.426752
SVM	0.958957	0.955882	0.223368	0.362117
Neural Network	0.971684	0.367647	0.892857	0.520833

<https://www.kaggle.com/46142589228756-ru-221-FeatureEngineering>

Из всех моделей лучшие результаты у Tuned Gradient Boosting.

- *Плюсы:* Выдает наивысший ROC-AUC, отлично ловит нелинейные зависимости (например, сочетание высокой мощности и температуры), не требует стандартизации данных.
- *Компромисс:* Обучается дольше, чем Логистическая регрессия, и работает как "черный ящик" (сложнее объяснить решение инженерам на заводе).

График ниже показывает, на что смотрит лучший алгоритм при принятии решения:

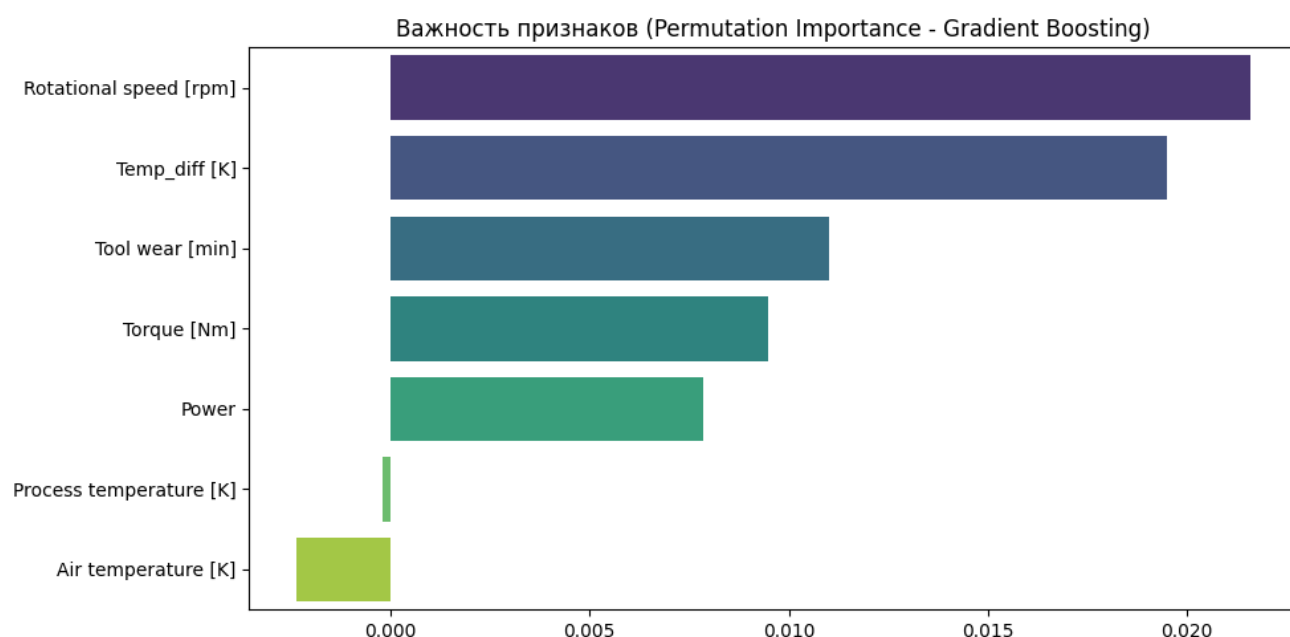
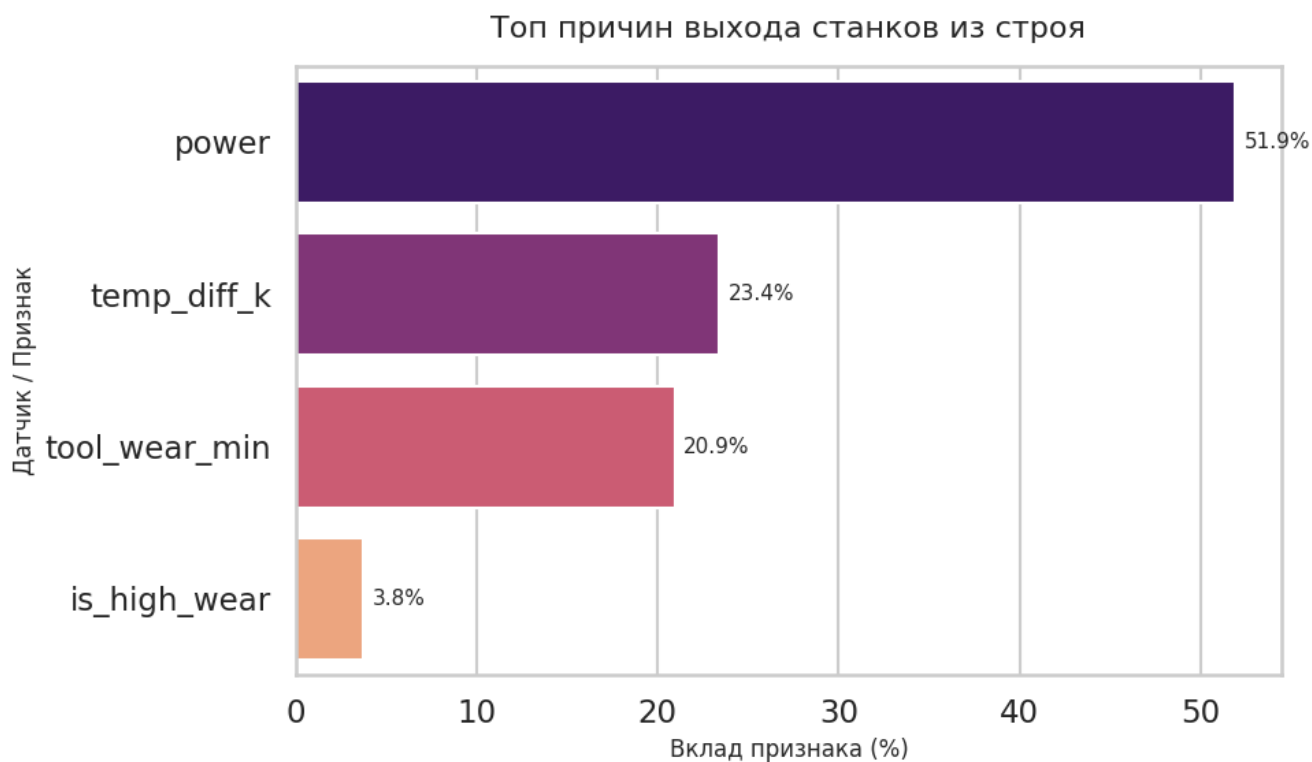


Рисунок 6: Важность признаков

- Rotational speed (Скорость вращения) и Temp_diff (Разница температур) — главные предикторы поломки.
- Базовые температуры сами по себе (воздуха и процесса) оказались почти бесполезными, что доказывает важность нашего Feature Engineering на этапе очистки.



6.2. Глубокая оценка лучшей модели

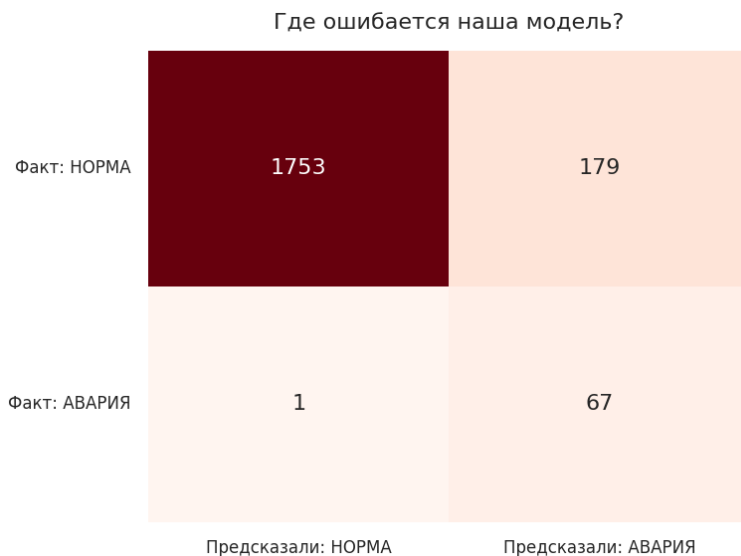


Рисунок 7: Матрица ошибок (Confusion Matrix)

Этот график показывает абсолютные значения правильных и неправильных предсказаний модели.

- Истинно отрицательные (1753): Модель верно распознала «Норму».
- Ложноположительные (179): Ложные тревоги. Модель предсказала «Аварию», хотя по факту была «Норма».

- Ложноотрицательные (1): Пропуск события. Модель предсказала «Норму», хотя по факту произошла «Авария».
- Истинно положительные (67): Модель успешно обнаружила «Аварию».

Главный вывод: Модель настроена на максимальную полноту (Recall). Она нашла 67 из 68 реальных аварий, пропустив всего одну. Однако за эту чувствительность приходится платить большим числом ложных срабатываний (179).

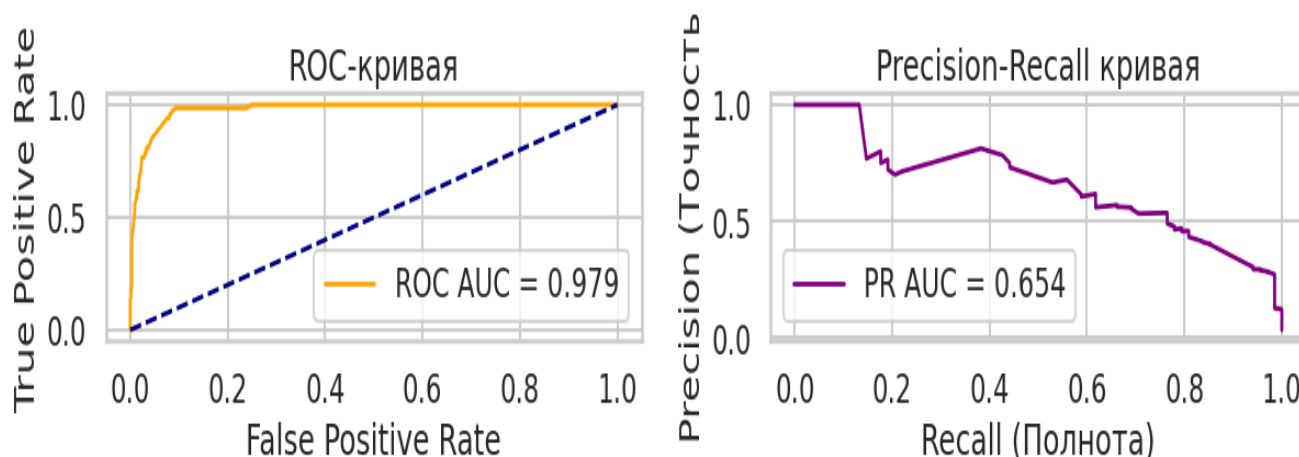


Рисунок 8: ROC-кривая и Precision-Recall

ROC-кривая: График показывает зависимость доли верно классифицированных аварий (True Positive Rate) от доли ложных срабатываний (False Positive Rate).

Метрика ROC AUC = 0.979 (близка к идеальной единице) говорит о том, что модель отлично разделяет два класса в целом.

Важное замечание: При сильном дисбалансе классов (как в вашем случае) ROC-кривая может выглядеть излишне оптимистично, так как огромное число примеров «Нормы» маскирует ошибки на малом классе.

Precision-Recall кривая: Этот график иллюстрирует баланс между Точностью (Precision — какая доля предсказанных аварий реально является авариями) и Полнотой (Recall — какую долю реальных аварий мы нашли).

Для задач с дисбалансом классов это самая показательная метрика.

PR AUC = 0.654 показывает более реалистичную картину. График идет вниз, потому что для того, чтобы найти почти 100% аварий (высокий Recall), модели приходится выдавать много ложных тревог, из-за чего стремительно падает Точность (Precision).

В целом, это хорошая модель для ситуации, когда пропустить аварию категорически нельзя, но есть ресурсы на проверку ложных сигналов.

6.3. Статистические тесты и А/В Симуляция

Tuned Gradient Boosting выигрывает у Baseline на пару процентов. Но это может быть случайность и чтобы доказать бизнесу, что модель реально работает, проведем статистический тест методом Bootstrap (создание 1000 случайных подвыборок) и симулируем А/В тест.

Статистический Bootstrap тест (1000 итераций)

- 95% Доверительный интервал Gradient Boosting: [0.968, 0.987]
- 95% Доверительный интервал Baseline: [0.826, 0.927]

Улучшение статистически значимо. Нижняя граница бустинга выше верхней границы Baseline ($p\text{-value} < 0.05$).

Симуляция А/В теста (Финансовый анализ) Представим, что:

- Ложный вызов бригады (False Positive) стоит заводу \$50
- Пропущенная авария станка (False Negative) обходится в \$5000 из-за замены деталей и простоя.

После проведения симуляции получаем:

- Убытки при использовании Базовой модели: \$53,450
- Убытки при использовании нашей ML модели: \$13,950

Экономия для бизнеса составит: \$39,500

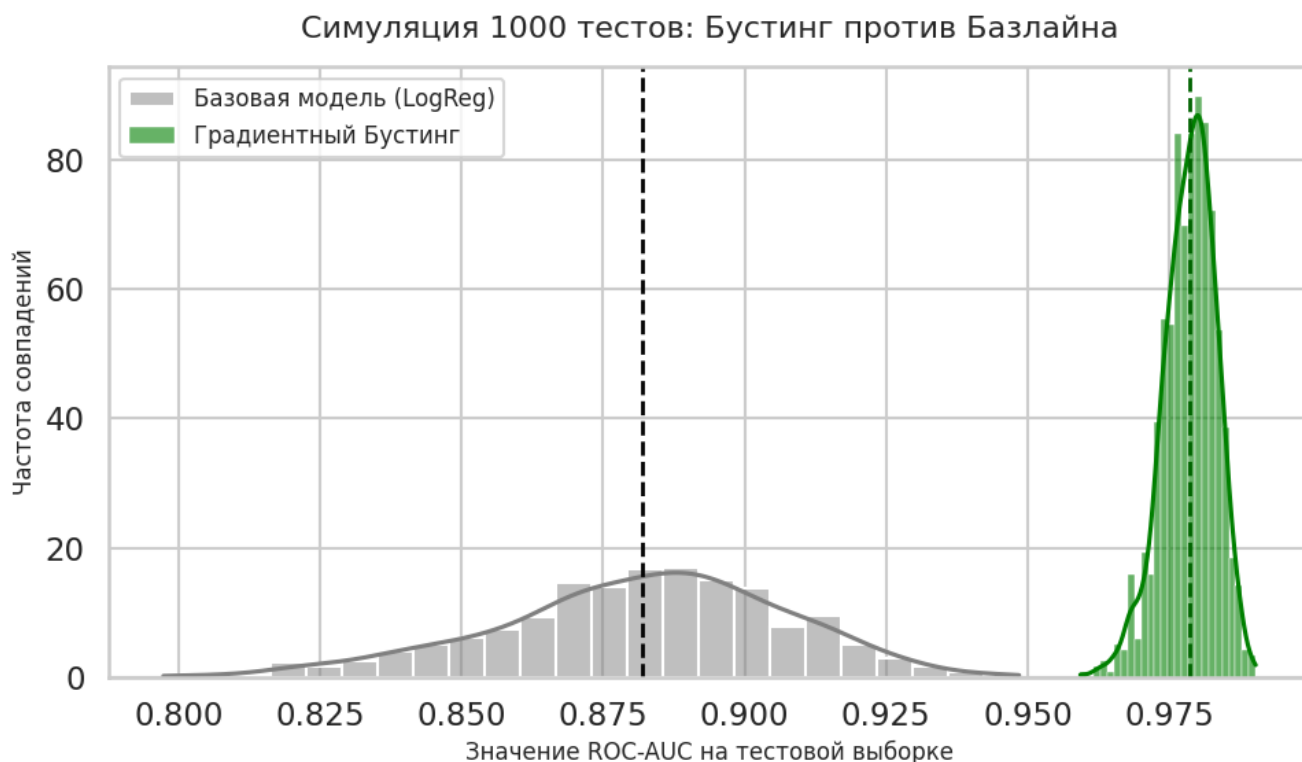


Рисунок 9: Симуляция А/В теста (Финансовый анализ)

Зеленый «холмик» (наша модель) находится правее серого (простая модель). Тот факт, что эти холмики почти не пересекаются, визуально доказывает стейкхолдерам: наше улучшение статистически значимо и не является случайностью.

Выводы

Пропуск реальной аварии обходится бизнесу или производству дорого, и значит текущая модель настроена верно. Она работает как высокочувствительный фильтр, который справляется со своей приоритетной целью. Пропущенная всего 1 авария (False Negative) против 67 найденных (True Positive). В условиях сильного дисбаланса классов получить такой высокий уровень полноты (Recall около 98.5%) — это прекрасный показатель для базовой версии алгоритма.

Как оптимизировать работу модели

Несмотря на успех в поиске аварий, модель выдает 179 ложных срабатываний (False Positives), на проверку которых операторам придется тратить время. Чтобы снизить эту нагрузку при дальнейшей разработке моделей, можно рассмотреть следующие технические решения:

- Двухэтапная фильтрация: Оставьте текущую модель как надежный первичный радар, который "ловит" все подозрительные события. Затем можно обучить вторую, более сложную ML-модель, которая будет классифицировать только отфильтрованные первой моделью случаи, отсекая ложные тревоги на основе более глубоких признаков.
- Тонкая настройка порога отсечки (Threshold Tuning): Проанализируйте вашу Precision-Recall кривую по точкам. Иногда можно минимально сдвинуть порог принятия решения, чтобы сократить количество ложных тревог (например, на 20-30 штук), сохранив при этом пропуск аварий на той же отметке в 1 случай.
- Cost-Sensitive Learning (Обучение с учетом стоимости ошибок): При переобучении можно задать асимметричные веса классов или матрицу штрафов, где штраф за пропуск аварии математически в десятки раз выше, чем штраф за ложную тревогу.
- Анализ ложноположительных примеров: Изучите срез данных с теми 179 примерами «Нормы», которые модель посчитала «Аварией». Часто там обнаруживаются шумные фичи, выбросы или ошибки разметки, которые можно исправить на этапе подготовки данных.